Presentation - Matgeo

Aryansingh Sonaye
AI25BTECH11032
EE1030 - Matrix Theory

September 18, 2025

Problem Statement

Problem 4.8.28

Find the equation of the plane passing through the intersection of the planes

$$\mathbf{r} \cdot (\hat{\imath} + \hat{\jmath} + \hat{k}) = 1, \tag{1.1}$$

$$\mathbf{r} \cdot (2\hat{\imath} + 3\hat{\jmath} - \hat{k}) + 4 = 0, \tag{1.2}$$

which is parallel to the X–axis. Hence, find the distance of the plane from the X–axis.

Description of Variables used

Quantity	Value
Normal of Π_1	$\mathbf{n}_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$
Right-hand side of Π_1	$b_1 = 1$
Normal of Π_2	$\mathbf{n}_2 = \begin{pmatrix} 2 \\ 3 \\ -1 \end{pmatrix}$
Right-hand side of Π_2	$b_2 = -4$
X–axis direction	$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$

Table: Input values for the problem

Theoretical Solution

We first find the line of intersection of the given planes. Stacking the two plane equations in matrix form gives

$$\mathbf{A}\mathbf{r} = \mathbf{b}, \tag{2.1}$$

$$\mathbf{A} = \begin{pmatrix} \mathbf{n}_1^\top \\ \mathbf{n}_2^\top \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 3 & -1 \end{pmatrix}, \tag{2.2}$$

$$\mathbf{b} = \begin{pmatrix} 1 \\ -4 \end{pmatrix}. \tag{2.3}$$

The augmented matrix is reduced as follows:

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 2 & 3 & -1 & -4 \end{array}\right) \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \left(\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & 1 & -3 & -6 \end{array}\right) \tag{2.4}$$

$$\xrightarrow{R_1 \leftarrow R_1 - R_2} \left(\begin{array}{ccc|c} 1 & 0 & 4 & 7 \\ 0 & 1 & -3 & -6 \end{array}\right). \tag{2.5}$$

Theoretical Solution

Thus the line of intersection can be expressed as

$$\mathbf{r} = \mathbf{r}_p + t\,\mathbf{d}, \tag{2.6}$$

$$\mathbf{r}_p = \begin{pmatrix} 7 \\ -6 \\ 0 \end{pmatrix}, \tag{2.7}$$

$$\mathbf{d} = \begin{pmatrix} -4 \\ 3 \\ 1 \end{pmatrix}. \tag{2.8}$$

Next, since the required plane is parallel to the X–axis, its normal $\mathbf{n}$ must be orthogonal to $\mathbf{e}_1$. Also, since the plane contains the intersection line, $\mathbf{n}$ must be orthogonal to $\mathbf{d}$. Hence the system is

$$\mathbf{e}_1^\top \mathbf{n} = 0, \tag{2.9}$$

$$\mathbf{d}^\top \mathbf{n} = 0. \tag{2.10}$$

Theoretical Solution

Writing this in matrix form:

$$\begin{pmatrix} \mathbf{e}_1^\top \\ \mathbf{d}^\top \end{pmatrix} \mathbf{n} = \begin{pmatrix} 1 & 0 & 0 \\ -4 & 3 & 1 \end{pmatrix} \mathbf{n} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \tag{2.11}$$

The augmented matrix reduces as

$$\left(\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ -4 & 3 & 1 & 0 \end{array}\right) \xrightarrow{R_2 \leftarrow R_2 + 4R_1} \left(\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & 3 & 1 & 0 \end{array}\right). \tag{2.12}$$

From this we conclude

$$n_x = 0, \tag{2.13}$$

$$3n_y + n_z = 0. \tag{2.14}$$

Theoretical Solution

Choosing a convenient scale, we may take

$$\mathbf{n} = \begin{pmatrix} 0 \\ -1 \\ 3 \end{pmatrix}. \tag{2.15}$$

Now the constant term of the plane is obtained by substituting a point $\mathbf{r}_p$ on the line:

$$c = \mathbf{n}^\top \mathbf{r}_p, \tag{2.16}$$

$$c = \begin{pmatrix} 0 & -1 & 3 \end{pmatrix} \begin{pmatrix} 7 \\ -6 \\ 0 \end{pmatrix}, \tag{2.17}$$

$$c = 6. \tag{2.18}$$

Therefore, the equation of the required plane is

$$\mathbf{n}^\top \mathbf{r} - c = 0, \tag{2.19}$$

$$\mathbf{n}^\top \mathbf{r} - 6 = 0. \tag{2.20}$$

Theoretical Solution

Finally, the distance of this plane from the X–axis is

$$d = \frac{|c|}{\|\mathbf{n}\|}, \tag{2.21}$$

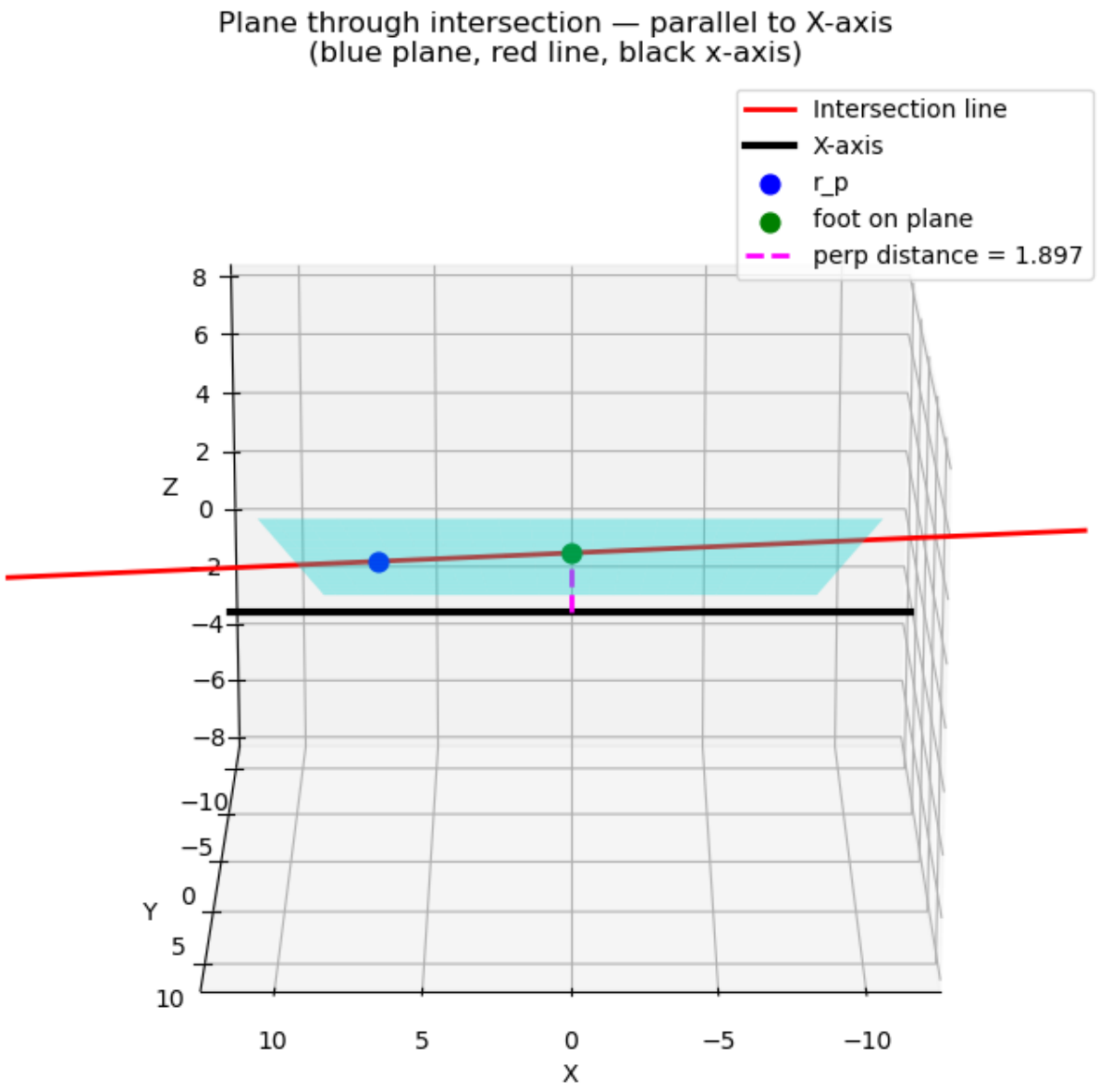
$$d = \frac{6}{\sqrt{\mathbf{n}^\top \mathbf{n}}}, \tag{2.22}$$

$$d = \frac{6}{\sqrt{10}}. \tag{2.23}$$

Final Answer

$$\boxed{\mathbf{n}^\top \mathbf{r} - c = 0}, \quad \boxed{\mathbf{n} = \begin{pmatrix} 0 \\ -1 \\ 3 \end{pmatrix}, \ c = 6}, \quad \boxed{\text{distance} = \frac{6}{\sqrt{10}}}. \tag{2.24}$$

Plot


Plane through intersection — parallel to X-axis
(blue plane, red line, black x-axis)
Intersection line
X-axis
r_p
foot on plane
perp distance = 1.897
Z
8
6
4
2
0
−2
−4
−6
−8
−10
Y
−5
0
5
10
X
10
5
0
−5
−10

Code - C

```
#include <math.h>

#define EPS 1e−9

// Row reduction to Reduced Row Echelon Form (RREF)
void row_reduce(int rows, int cols, double *M) {
    int r = 0; // pivot row
    for (int c = 0; c < cols − 1 && r < rows; c++) {
        // 1. Find pivot
        int pivot = r;
        for (int i = r+1; i < rows; i++) {
            if (fabs(M[i*cols + c]) > fabs(M[pivot*cols + c])) {
                pivot = i;
            }
        }
        if (fabs(M[pivot*cols + c]) < EPS) continue;
```

Code - C

```
// 2. Swap rows if needed
if (pivot != r) {
    for (int j = 0; j < cols; j++) {
        double tmp = M[r*cols + j];
        M[r*cols + j] = M[pivot*cols + j];
        M[pivot*cols + j] = tmp;
    }
}

// 3. Normalize pivot row
double div = M[r*cols + c];
for (int j = c; j < cols; j++) {
    M[r*cols + j] /= div;
}
```

Code - C

```
        // 4. Eliminate in all other rows
        for (int i = 0; i < rows; i++) {
            if (i == r) continue;
            double factor = M[i*cols + c];
            for (int j = c; j < cols; j++) {
                M[i*cols + j] -= factor * M[r*cols + j];
            }
        }
        r++;
    }
}
```

Code - Python(with shared C code)

The code to obtain the required plot is

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt

# Load C shared library
lib = ctypes.CDLL("./librowreduce.so")
lib.row_reduce.argtypes = (ctypes.c_int, ctypes.c_int,
                           ctypes.POINTER(ctypes.c_double))
lib.row_reduce.restype = None
```

Code - Python(with shared C code)

```
def row_reduce_numpy(A):
    """Run row_reduce from C on numpy array A (augmented matrix)."""
    rows, cols = A.shape
    A_flat = (ctypes.c_double * (rows*cols))(*A.flatten())
    lib.row_reduce(rows, cols, A_flat)
    return np.array(A_flat).reshape(rows, cols)


# Problem setup
n1 = np.array([1,1,1], dtype=float)
b1 = 1.0
n2 = np.array([2,3,-1], dtype=float)
b2 = -4.0
e1 = np.array([1,0,0], dtype=float)
```

Code - Python(with shared C code)

```
# Step 1: Line of intersection
M_line = np.array([[*n1, b1],
                   [*n2, b2]], dtype=float)
R_line = row_reduce_numpy(M_line)

# Particular point rp and direction d
rp = np.array([R_line[0,3], R_line[1,3], 0]) # set z=0 for particular
    solution
d = np.array([-R_line[0,2], -R_line[1,2], 1]) # free variable = z

print("Particular point r_p =", rp)
print("Direction d =", d)

# Step 2: Normal vector
M_normal = np.array([[*e1, 0],
                     [*d, 0]], dtype=float)
R_normal = row_reduce_numpy(M_normal)
```

Code - Python(with shared C code)

```
# Extract n: last variable free = 1
n = np.array([-R_normal[0,2], -R_normal[1,2], 1])
c = np.dot(n, rp)

print("Normal vector n =", n)
print("Plane constant c =", c)

# Step 3: Distance from x-axis
p = np.array([0.0,0.0,0.0]) # any point on x-axis
dist = abs(np.dot(n,p) - c) / np.linalg.norm(n)
print("Distance from x-axis =", dist)

# Step 4: Plot figure
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

# Line of intersection
```

Code - Python(with shared C code)

```
t_vals = np.linspace(-5, 5, 100)
line_pts = np.array([rp + t*d for t in t_vals])
ax.plot(line_pts[:,0], line_pts[:,1], line_pts[:,2],
        color="red", lw=2, label="Line-of-intersection")

# Plane surface (n = [0,-1,1] , solve for Y in terms of X,Z)
x_vals = np.linspace(-5, 10, 30)
z_vals = np.linspace(-6, 6, 30)
X, Z = np.meshgrid(x_vals, z_vals)
Y = (c - n[0]*X - n[2]*Z)/n[1]
ax.plot_surface(X, Y, Z, alpha=0.5, color="blue")

# X-axis
ax.plot([-10, 10], [0, 0], [0, 0], 'k-', lw=3, label="X-axis")
```

Code - Python(with shared C code)

```
# Formatting
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)
ax.set_zlim(-6, 6)
ax.view_init(elev=20, azim=45) # adjust angle to see parallelism
ax.legend()
plt.savefig("plane.png")
plt.show()
```

Code - Python only

```
import numpy as np
import matplotlib.pyplot as plt

# Gauss−Jordan row reduction with partial pivoting (returns RREF)
# A is a numpy array (rows x cols), operates on a copy
def row_reduce(A_in, eps=1e−12):
    A = A_in.astype(float).copy()
    rows, cols = A.shape
    r = 0
    for c in range(cols − 1): # last column is RHS
        if r >= rows:
            break

        # partial pivot: find row with max abs in column c at or below r
        pivot = r
        maxv = abs(A[pivot, c])
```

Code - Python only

```
for i in range(r+1, rows):
    v = abs(A[i, c])
    if v > maxv:
        maxv = v
        pivot = i
if maxv < eps:
    # no pivot in this column
    continue
# swap if needed
if pivot != r:
    A[[r, pivot], :] = A[[pivot, r], :]
# normalize pivot row
div = A[r, c]
A[r, c:] = A[r, c:] / div
# eliminate in all other rows
```

Code - Python only

```
        for i in range(rows):
            if i == r:
                continue
            factor = A[i, c]
            if abs(factor) > eps:
                A[i, c:] = A[i, c:] - factor * A[r, c:]
        r += 1
    # zero-out tiny values for neatness
    A[np.abs(A) < 1e-12] = 0.0
    return A

# Problem data
n1 = np.array([1.0, 1.0, 1.0])
b1 = 1.0
n2 = np.array([2.0, 3.0, -1.0])
b2 = -4.0
e1 = np.array([1.0, 0.0, 0.0]) # x-axis direction
```

Code - Python only

```
# Step 1: line of intersection [A|b]
M_line = np.vstack([np.hstack([n1, b1]), np.hstack([n2, b2])])
R_line = row_reduce(M_line)
print("RREF (line system):\n", R_line)

# Interpret RREF: assume last variable (z) free -> set z = t
# Row form: row0: x + a z = r0 => x = r0 - a t
# row1: y + b z = r1 => y = r1 - b t
rp = np.array([R_line[0,3], R_line[1,3], 0.0]) # t=0
d = np.array([-R_line[0,2], -R_line[1,2], 1.0]) # coefficients of t
print("\nParticular point r_p =", rp)
print("Direction d =", d)
```

Code - Python only

```
# Step 2: plane normal: solve [e1^T; d^T] n = 0
M_norm = np.vstack([np.hstack([e1, 0.0]), np.hstack([d, 0.0])])
R_norm = row_reduce(M_norm)
print("\nRREF (normal system):\n", R_norm)

# choose last variable free = 1 -> n = [-R[0,2], -R[1,2], 1]
n = np.array([-R_norm[0,2], -R_norm[1,2], 1.0])
# scale normal to an integer-like/clean vector for nicer printing (optional
    )
# avoid scaling if zero
if np.linalg.norm(n) > 1e-12:
    # scale to make components integers if possible (here we know
        integers)
    # but we won't force scale; keep as computed
    pass
```

Code - Python only

```
c = float(np.dot(n, rp))
print("\nNormal n =", n)
print("Plane constant c =", c)

# Step 3: distance from x-axis
# Choose a convenient point on x-axis p0 = (0,0,0)
# distance = | n.p0 - c | / ||n|| = | -c | / ||n||
# Also compute the perpendicular foot point from p0 onto plane:
# foot = p0 + ((c - n.p0)/||n||^2) * n
p0 = np.array([0.0, 0.0, 0.0])
n_norm = np.linalg.norm(n)
if n_norm < 1e-12:
    dist = float('nan')
    foot = None
else:
    dist = abs(np.dot(n, p0) - c) / n_norm
    foot = p0 + ((c - np.dot(n, p0)) / (n_norm**2)) * n
```

Code - Python only

```
print("\nDistance from x-axis =", dist)
print("Perpendicular foot on plane from origin (x-axis point) =", foot)

# Step 4: Plot everything clearly showing plane parallel to x-axis
fig = plt.figure(figsize=(9,7))
ax = fig.add_subplot(111, projection='3d')

# plot line of intersection (parametric)
t_vals = np.linspace(-6, 6, 201)
line_pts = np.array([rp + t*d for t in t_vals])
ax.plot(line_pts[:,0], line_pts[:,1], line_pts[:,2], color='red', lw=2, label='
    Intersection line')
```

Code - Python only

```
# Create plane grid such that plane visibly looks parallel to x−axis:
if abs(n[1]) > 1e−9:
    # make grid in X and Z (X free direction shows parallelism)
    x_vals = np.linspace(−10, 10, 40)
    z_vals = np.linspace(−8, 8, 40)
    X, Z = np.meshgrid(x_vals, z_vals)
    Y = (c − n[0]∗X − n[2]∗Z) / n[1]
    ax.plot_surface(X, Y, Z, alpha=0.35, color='cyan', rstride=2, cstride
        =2, linewidth=0)
else:
    # if n[1] ˜ 0, solve for X instead (rare)
    y_vals = np.linspace(−10, 10, 40)
    z_vals = np.linspace(−8, 8, 40)
    Y, Z = np.meshgrid(y_vals, z_vals)
    X = (c − n[1]∗Y − n[2]∗Z) / n[0]
    ax.plot_surface(X, Y, Z, alpha=0.35, color='cyan', rstride=2, cstride
        =2, linewidth=0)
```

Code - Python only

```
# plot x−axis bold
ax.plot([−12, 12], [0,0], [0,0], color='k', lw=3, label='X−axis')

# mark particular point rp and foot
ax.scatter([rp[0]],[rp[1]],[rp[2]], color='blue', s=60, label='r_p')
if foot is not None:
    ax.scatter([foot[0]],[foot[1]],[foot[2]], color='green', s=60, label='foot
        -on-plane')
    # draw perpendicular segment from p0 on x−axis to foot
    ax.plot([p0[0], foot[0]], [p0[1], foot[1]], [p0[2], foot[2]],
            color='magenta', lw=2, linestyle='−−', label=f'perp-
                distance-=-{dist:.3f}')

# labels and view − choose view that shows parallelism (look along x to
    show plane side)
```

Code - Python only

```
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_xlim(-12, 12)
ax.set_ylim(-12, 12)
ax.set_zlim(-8, 8)

# set viewing angle so x-axis appears lateral to plane: azim 90 looks
    down +x direction
ax.view_init(elev=18, azim=90)
ax.legend()
ax.set_title("Plane through intersection - parallel to X-axis\n(blue plane,
    red line, black x-axis)")
plt.tight_layout()
plt.savefig("new_plane.png")
plt.show()
```